

How We Build the Prepayment WAL — Code & Data Walkthrough

For amortizing lending products — home loans, business loans — a real contractual maturity date exists, but it systematically overstates how long the loan actually stays on our book. So rather than trusting that date as-is, we look up an already-adjusted weighted average life. See `docs/behavioural_wal.md` for why this product type needs its own treatment; here's the lookup itself, as we implement it in `lookup_prepayment_wal`.

The lookup, tier by tier

Every `PREPAYMENT`-type account resolves its `CALCULATED_WAL` from a dedicated reference sheet, and we try it at two levels of specificity:

1. **(Branch, AC_CAT)** — our most specific match. `AC_CAT` is the account-category code (a specific home-loan or business-loan product code, for instance); we prefer a direct match at this level whenever the reference sheet has one.
2. **(Branch, ALCO_CLASS)** — a broader fallback we only use when tier 1 comes up empty. Rather than picking an arbitrary single `AC_CAT`'s figure, we take the **mean** `FINAL_PREPAYMENT_ADJUSTED_WAL` across every `AC_CAT` that rolls up to the same `ALCO_CLASS` at that branch — a genuine broader aggregate, not a first-match guess. This matters for a genuinely new or reclassified product code the reference sheet hasn't caught up with yet: it still gets a sensible branch-level figure instead of falling all the way through to the raw contractual tenor.

We built this as its own dedicated tier rather than reusing our general-purpose fallback mechanism (which we use elsewhere for simple value substitution) — matching a broader key entirely is a different kind of operation from substituting one exact value for another, and encoding "try a broader key" as a fallback-sheet data row would blur that distinction.

Example

ACC_NO	WAL_TYPE	CURRENT_AVG_CONTRACTUAL_WAL	CALCULATED_WAL
HOME001	PREPAYMENT	12.4	7.85

The gap — 12.4 years of contractual maturity against a 7.85-year adjusted figure — is the quantified effect of customers refinancing, selling, and settling early. Pricing this loan book at its own contractual 12.4 years would price it as if none of that behavior exists.

The starting methodology tier

We build the adjusted WAL itself from a roll-rate style prepayment model — observed early-settlement behavior converted into an expected life shorter than the contractual schedule implies. We recommend a **Two-Point Roll Rate** approach (comparing scheduled vs. actual outstanding balance at two points in time to infer an implied prepayment rate) as the starting tier for anyone building this out for the first time — simple enough to validate by hand, before layering in more granular vintage- or rate-sensitivity-based models later.

Where we hit real limits

- **An adjusted WAL should never exceed the contractual WAL** — prepayment can only shorten expected life, never lengthen it. We treat this as a useful sanity check to build into any implementation: if `FINAL_PREPAYMENT_ADJUSTED_WAL > CURRENT_AVG_CONTRACTUAL_WAL` for any row, that's a data or aggregation bug, not a modeling outcome. We've found it's worth watching for specifically when combining figures across branches — averaging columns that were filtered inconsistently by branch (one column branch-filtered, another not) is a realistic way to accidentally violate this invariant even though each underlying figure is individually correct. A dedicated reconciliation check ($\text{adjusted} \leq \text{contractual}$, every row, every branch) is cheap insurance.
- **Our reference sheet needs to stay current.** Like the structural-split reference data (`docs/persistence_structural_split_wal.md`), this lookup is only as good as how up to date the prepayment reference sheet is — a new product code needs an entry (or at least a correct `ALCO_CLASS` rollup) before it prices sensibly.

See also `docs/behavioural_wal.md`.